This section's header gives away the solution: automated testing. You write code to test your code. Learning how to write good, automated tests is a skill in itself and an important part of working as a developer. It isn't uncommon to find a 1:1 relationship between lines of code and lines of tests in larger organizations with a high degree of automation. That means you'll be spending half your time writing tests.

8.1.1 The scope of tests

Let's touch on a bit of background before diving into how Python and Django structure automated tests. Automated testing is usually broken down into categories. The category definitions are a little loose, and you'll find different developers split them at different points. Figure 8.1 shows a four-category version of the testing pyramid.

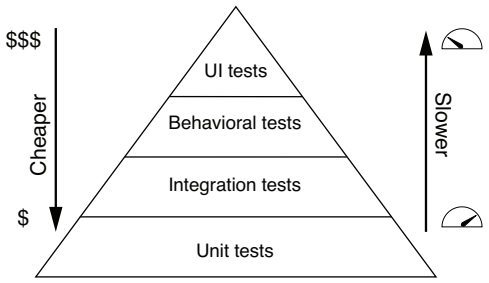


Figure 8.1 Hierarchy of testing categories

If you search around on the internet, you'll find several versions of this diagram, ranging from three levels to seven levels and with different names for each category. The most well-defined category is the one on the bottom: unit tests. A *unit test* is a test performed on a single component of software. This can be vague though, as two people might not agree on the definition of "component." Typically, a unit test is at the level of a single function.

You want your unit level to be as independent of other parts of your software as possible and with as few side effects as possible. If a series of tests gets done at the unit level, it shouldn't matter what order they run in; you should always get the same result. This can get tricky because writing to the database is a side effect. I'll cover this in more detail later in the chapter.

The next level up in the testing pyramid is *integration tests*. These are tests that operate over several units. The purpose here is to ensure that when the different pieces work together, you still get the desired result. This level is sometimes also known as *system tests*. In software, that has a graphical component: the integration typically is still only at the business logic or server level, not the GUI.

Behavioral tests are those that map to the requirements. You, or someone on your software team, has decided that a certain feature should exist and perform some action. A behavioral test ensures the feature is working as designed. These are also known as *acceptance tests*, named after Acceptance Criteria, which are part of a User Story, a common way of writing requirements in Agile development methodologies. This category of testing has become quite popular, as there are frameworks out there which specifically test at this level. One of the most popular is Cucumber (https://cucumber.io/).

At the top level of the pyramid, where you might be expecting an ominous looking eye, are *UI tests*. The name more or less gives it away: these are tests on the user